



University
of Glasgow

W4-05: Advantages and Disadvantages with Spark

COMPSCI4064 (H) and COMPSCI5088 (M)

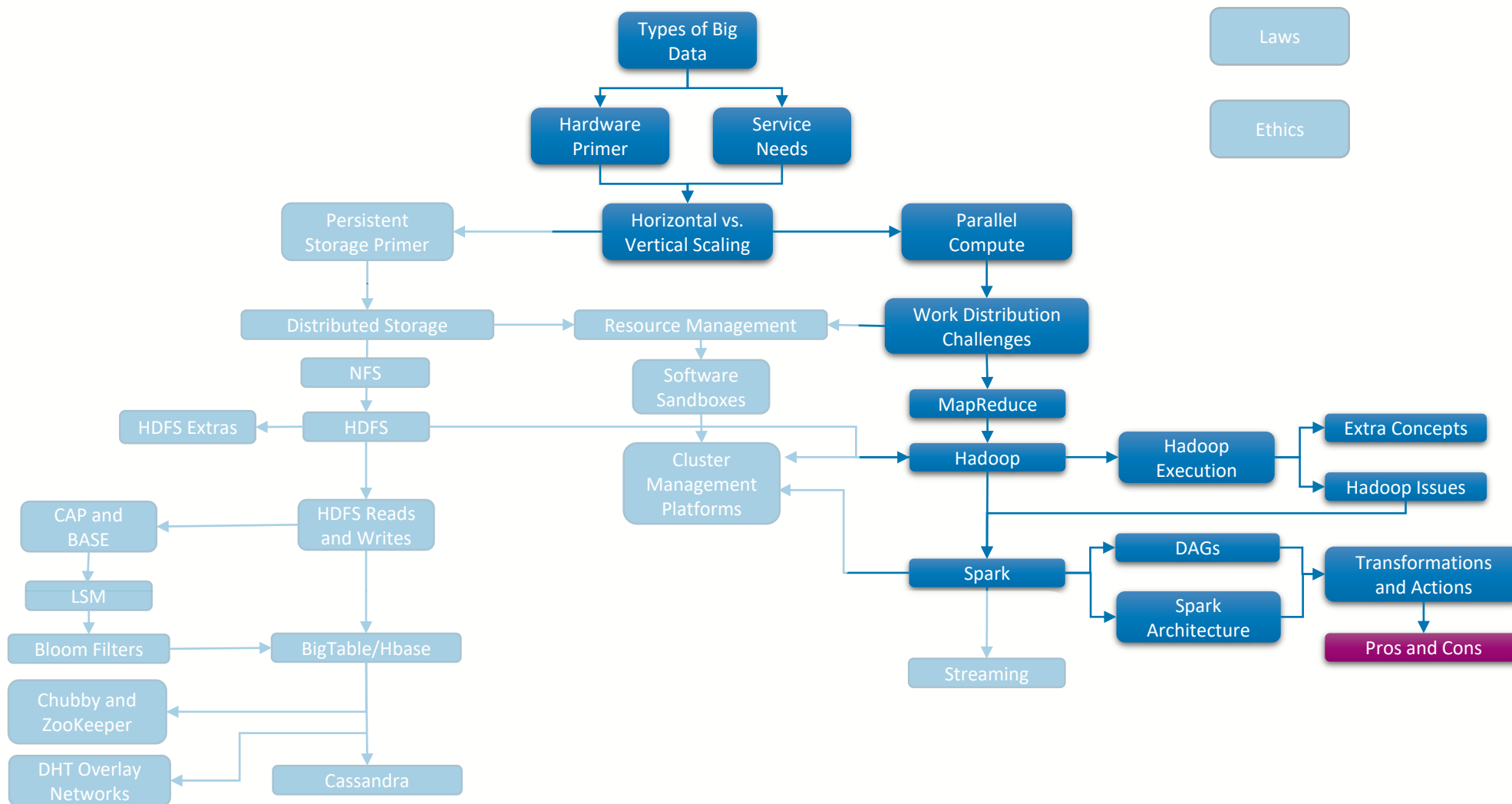
Dr. Richard McCreddie

WORLD
CHANGING
GLASGOW

A WORLD
TOP 100
UNIVERSITY



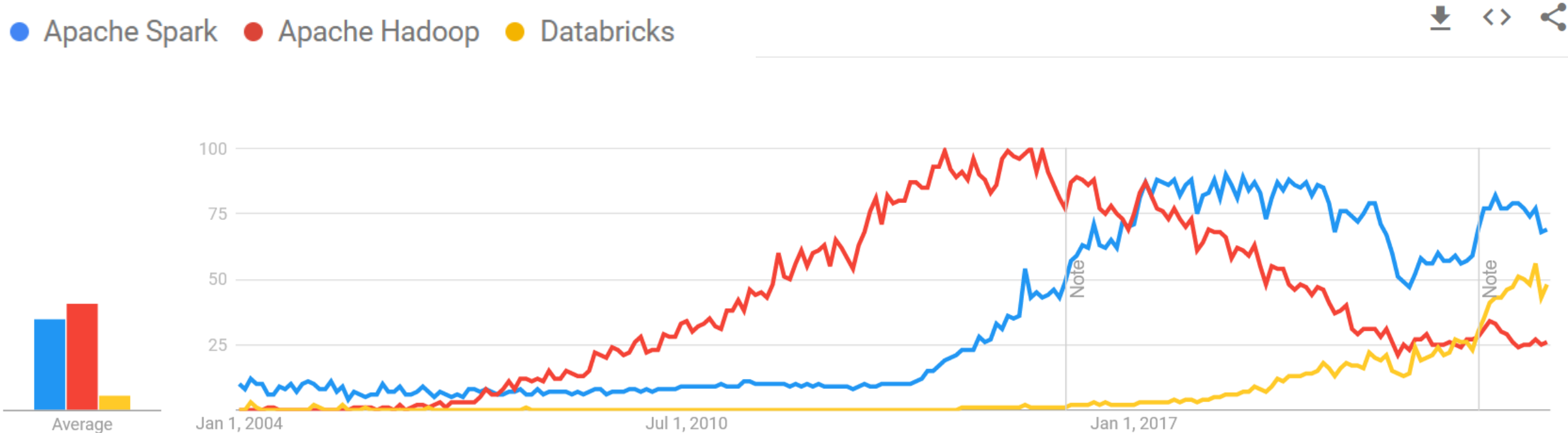
Where Are We?





Spark Popularity

- Since around 2014, Apache Spark exploded in popularity, and overtook Hadoop in 2017
- **What has it solved, and what are the issues remaining?**





Speed

- We have covered this already, but Spark's biggest call to fame is its speed in comparison to Hadoop.
 - Its often reported that Spark is upto 100x faster
 - Largely because all the computation is in-memory

	Hadoop MR Record	Spark Record	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400 physical	6592 virtualized	6080 virtualized
Cluster disk throughput	3150 GB/S (est.)	618 GB/S	570 GB/S
Sort Benchmark Daytona Rules	Yes	Yes	No
Network	dedicated data center, 10Gbps	virtualized (EC2), 10Gbps network	virtualized (EC2), 10Gbps network
Sort rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Sort rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min



Wider Use-case Coverage

- As you have seen, Spark supports a much wider range of transformations than Hadoop, meaning that it can handle a larger number of use-cases
 - Most notably complex pipelines that require chaining many operations

Transformation	Meaning
map (<i>func</i>)	Return a new distributed dataset formed by passing each element of the source through a function <i>func</i> .
filter (<i>func</i>)	Return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true.
flatMap (<i>func</i>)	Similar to map, but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item).
mapPartitions (<i>func</i>)	Similar to map, but runs separately on each partition (block) of the RDD, so <i>func</i> must be of type <code>Iterator<T> => Iterator<U></code> when running on an RDD of type T.
mapPartitionsWithIndex (<i>func</i>)	Similar to mapPartitions, but also provides <i>func</i> with an integer value representing the index of the partition, so <i>func</i> must be of type <code>(Int, Iterator<T>) => Iterator<U></code> when running on an RDD of type T.
sample (<i>withReplacement</i> , <i>fraction</i> , <i>seed</i>)	Sample a fraction <i>fraction</i> of the data, with or without replacement, using a given random number generator seed.
union (<i>otherDataset</i>)	Return a new dataset that contains the union of the elements in the source dataset and the argument.
intersection (<i>otherDataset</i>)	Return a new RDD that contains the intersection of elements in the source dataset and the argument.
distinct ([<i>numPartitions</i>])	Return a new dataset that contains the distinct elements of the source dataset.
groupByKey ([<i>numPartitions</i>])	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using <code>reduceByKey</code> or <code>aggregateByKey</code> will yield much better performance. Note: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional <i>numPartitions</i> argument to set a different number of tasks.
reduceByKey (<i>func</i> , [<i>numPartitions</i>])	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> , which must be of type <code>(V,V) => V</code> . Like in <code>groupByKey</code> , the number of reduce tasks is configurable through an optional second argument.
aggregateByKey (<i>zeroValue</i>)(<i>seqOp</i> , <i>combOp</i> , [<i>numPartitions</i>])	When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a

Better Language and Integration Support

- Hadoop is wholly Java based, with support added later for Python
- Spark is written in Scala, with equivalent library support provided in Java
 - However, Spark has much better secondary language support, with plugins for Python and R, as well as native SQL query support
- Spark also integrates into a wide range of data science and storage platforms

Data science and Machine learning



SQL analytics and BI



Storage and Infrastructure





Spark as the Golden Bullet?

- Apache Spark solves most of the issues that plagued Hadoop, while maintaining the simplicity of code development
 - It's the **best general big data framework**
- However, Spark is not always the correct answer, for some types of specialist applications there is a better solution
 - The big use-cases here are **big data streaming** and associated to that **stateful processing**
 - In these areas, while Spark provides some support (via spark streaming), its not as fast or efficient as a dedicated platform like **Apache Flink**



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow